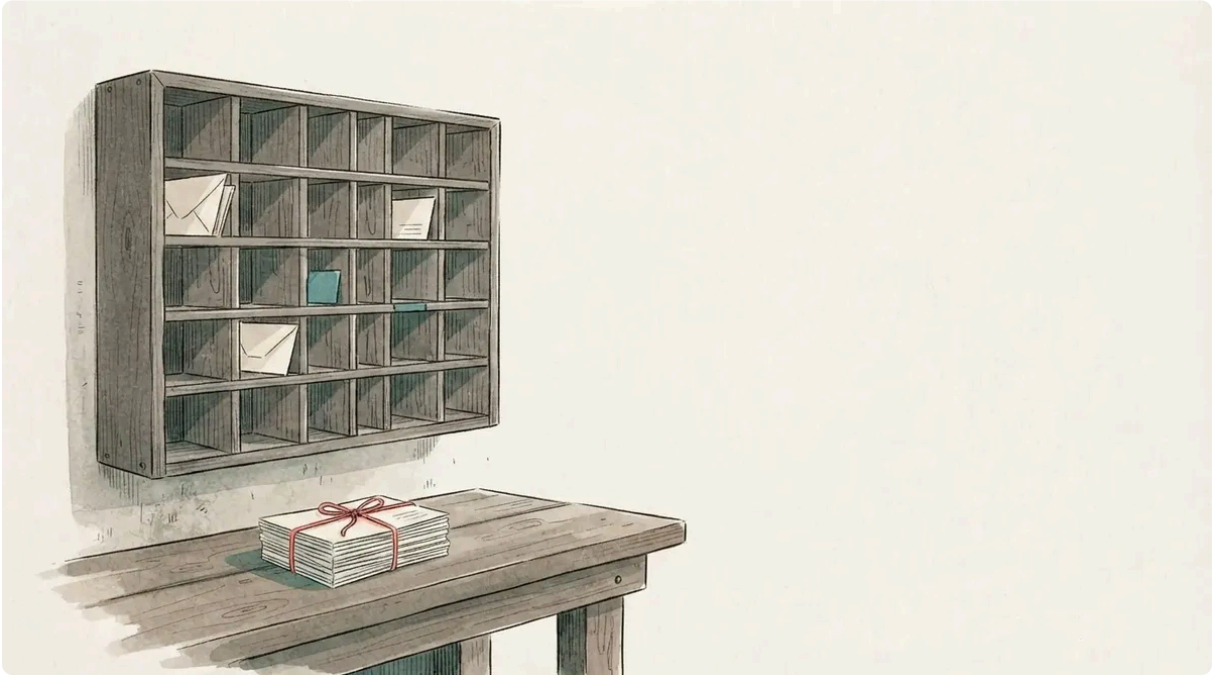
Self-hosting email is not that hard anymore

Emil Lindfors | February 18, 2026



Type your address into the box at the bottom of this page and here is what happens:

1. A Rust Worker on Cloudflare checks the address, checks a rate limit, and mails you a signed link. It does not store your address anywhere.
2. You open the mail and press a button. That button posts back to the Worker, which checks the signature and only then adds you to a mailing list on my own mail server.
3. A welcome mail arrives with the five most recent posts in it.
4. When I publish something, one message is written for you alone, with an unsubscribe link that knows who you are, and sent through that same server.
5. If your mail client offers a native unsubscribe button, it posts to the Worker, which removes you. The link in the footer shows a button that does the same.

No database, no newsletter platform, no third party holding the list. The mail server is a $6 VPS that also does my personal email. This is the February 2026 system after a year of fixes, and it is not the system any more: on 3 September 2026 I retired the Worker and moved the whole newsletter onto the mail server with a Postgres behind it, in one day. That move has its own post, and the rest of this one describes the system as it stood, because the mail-server half is unchanged and the reasons for the Worker half still explain the shape it took. This post is what it looks like now, what it needs on the DNS side to stay out of junk folders, and why I bothered.

Why a newsletter

Newsletters are a good way to reach people now that I don't want to be on social media much and still want to convey what I have to say to an audience that wants to hear it.

That is most of the reason. The rest is the part of my PhD that was about how knowledge moves between people, because a newsletter is a very clean example of it.

Bathelt, Malmberg and Maskell (2004) asked why firms cluster in the same city when the internet lets them talk to anyone. Their answer was that knowledge travels two ways. *Local buzz* is what you pick up by being in the room: gossip, overheard problems, who is hiring, what did not work. You do not choose it and you cannot switch it off. *Global pipelines* are the opposite: deliberate channels to specific people far away, chosen on both ends, built and maintained on purpose, carrying less, but carrying it to people outside your own room. The classic example is a Silicon Valley firm's standing relationship with a lab in Cambridge. Buzz keeps a cluster coherent; pipelines are where the new ideas come in.

Social media is buzz at planetary scale, with an algorithm deciding which room you are in. A newsletter is a pipeline. The reader chose it, I chose what goes down it, it carries one post a fortnight, and the whole point is to reach people outside my own cluster. An older result says why that matters: new information tends to arrive through weak ties, not close ones (Granovetter, 1973), and a stranger who reads your writing is the weakest tie there is.

The paper is about firms and regions, and I am one person with a blog, so read the transfer as a way of thinking and not as a finding. But it does make a prediction: the value of a pipeline is in the few people who write back, not in the count.

No real purpose other than to attract other like-minded people to get in touch.

The parts

Three components, all of them things I already had:

Part	What it does	Where
Zola static site	The blog, the subscribe form, and a plain-markdown copy of each newsletter issue at `/newsletter/<slug>.md`	Cloudflare Pages
Rust Worker	`/api/subscribe`, `/api/confirm`, `/api/unsubscribe`, `/api/send-newsletter`	Cloudflare Workers, free tier

Part	What it does	Where
Stalwart mail server	The subscriber list, delivery, and two WebDAV folders of logs	A Hetzner VPS, shared with everything else

The Worker is workers-rs, compiled to WASM. It started at about 700 lines in February. It is about 3,100 lines and 52 tests now, and most of the growth is the difference between a thing that works for me and a thing that can be pointed at strangers.

The mail server does the work a newsletter platform would do, through two of its protocols:

- **JMAP** for everything the Worker does. The subscriber list is a Stalwart *MailingList* object with a `recipients` map, `{"someone@example.com": true}`. Subscribing is one `x:MailingList/set` call that sets `recipients/<addr>` to `true`; unsubscribing sets it to `null`. Sending is an `Email/set` plus an `EmailSubmission/set` in one request.
- **WebDAV** for the logs. One folder holds one file per sent issue, and one holds one file per subscribe, confirm and unsubscribe event, with the address stored as an HMAC so the log can answer "did this person consent, and when" without being a list of former subscribers.

That last part is what "no database" means now: one file per fact, on a server the whole system already depends on, written with plain HTTP. Every fact this system holds is a JSON file you can list with `PROPFIND` and read with `GET`, and the Worker never needs a transaction across two of them.

Subscribe: the pending subscriber lives in the link

Double opt-in is normally where "no database" dies, because you need somewhere to keep the addresses that have asked but not confirmed. Except that the pending state is only `(address, deadline)`, and both fit in the confirmation link if the server can tell its own links from forged ones:

```
fn confirm_signature(secret: &str, email: &str, exp: u64) -> String {
    sign(secret, &format!("confirm:v1:{}:{}", exp, email))
}
```

`/api/subscribe` mails that link and touches nothing. `/api/confirm` recomputes the signature, checks the expiry, and then does the JMAP call. The link is good for 48 hours, and the click is itself the demonstrable consent, which is what GDPR asks for. The four decisions inside that one function, and why the confirmation is a button and not the link itself, are in the one-click unsubscribe post. Read that one before you copy this one.

Send: one message per reader

The February version of this post sent one message to the list address and let Stalwart fan it out to everyone. That was cheaper and it had to go, because the one-click unsubscribe standard, RFC 8058, needs the unsubscribe URL in each message to identify the reader, and a fanned-out message carries the same headers for everyone. So the Worker now reads the `recipients` map and sends once per address:

```
for email in &recipients {
    let unsubscribe_url =
        unsubscribe_link(&site_url, email, &unsubscribe_signature(&secret,
email));
    let html = email_template(/* ... */, &unsubscribe_url);

    match jmap_send_email(&sender, email, &subject, &html,
Some(&unsubscribe_url)).await {
        Ok(()) => sent += 1,
        Err(e) => {
            console_error!("Newsletter send to {} failed: {}", email, e);
            failed.push(email.clone());
        }
    }
}
```

Each message carries `List-Unsubscribe` with a signed per-reader URL, `List-Unsubscribe-Post: List-Unsubscribe=One-Click`, and the same URL in the footer. Three things around that loop are worth copying:

- **The slug is claimed before anyone is mailed.** A `PUT` with `If-None-Match: *` to `<send-log>/<slug>.json`. Stalwart answers 412 if the file exists, so a second send of the same issue is refused with a 409 before a single message goes out. If the log server cannot be reached, the send fails. It fails closed on purpose: mailing everyone twice is worse than mailing nobody.
- **Above 45 recipients the send is refused, not truncated.** (Gone with the Worker; the service on the box has no such cap.) Every message is a subrequest and Workers caps those at 50 per invocation on the free plan. Mailing the first 45 and reporting success is the worst available outcome, so past that it needs batching, and I have not needed it.
- **A partial send says so.** `{"sent": N, "failed": [...]}` with a 502, so a human looks at it.

The workflow from my side is two commands. `site-tools newsletter gen <post>` writes the issue as markdown with a small YAML header into `static/newsletter/`, the site deploys, and `site-tools newsletter send <slug>` calls the Worker with a bearer token. The Worker fetches the markdown from the live site, renders it with `pulldown-cmark`, and wraps it in a hardcoded HTML template with inline styles, because email clients. It renders fine in Gmail, Apple Mail and Outlook.

The mail side, which is the part people warn you about

Before, I have been scared about DKIM and have gotten issues with having been in a junk folder because of not having everything in order on the mail side, but now I have fixed DNSSEC, DKIM, DANE and all the bells and whistles so that we should have proper delivery, and from the initial testing it seems to work fine.

In February this domain had SPF, DKIM and DMARC and nothing else, and mail from it had been landing in junk folders. “Everything in order” is a list, and every item on it is a DNS record you can check from anywhere. I looked every one of these up from Cloudflare’s resolver while writing this section, and all of them came back DNSSEC-validated:

Record	What it says	Why it is there
`DS` at the registrar	DNSSEC, algorithm 13	Every record below is signed, so a receiver can trust the rest
`TXT` at the apex	`v=spf1 mx -all`	Only the MX host may send for this domain
`TXT` at `202412r._domainkey` and `202412e._domainkey`	An RSA key and an ed25519 key	Two DKIM signatures on every message
`TXT` at `_dmarc`	`v=DMARC1; p=reject; rua=mailto:postmaster@…`	Anything failing SPF and DKIM is rejected, and I get the reports
`TXT` at `_mta-sts` plus a policy file over HTTPS	`mode: enforce`, `mx: mail.lindfors.no`	Senders must use TLS to reach me
`TXT` at `_smtp._tls`	`v=TLSRPTv1; rua=mailto:…`	Reports when that TLS fails
`TLSA` at `_25._tcp.mail`	Four records, three pinning the CA chain and one the leaf	DANE: the certificate is pinned in signed DNS
`PTR` for the server’s IP	`mail.lindfors.no`	Reverse DNS matches the hostname it announces

Two of those took real work to get right. The first was the DKIM `h=` tag, which names the headers a signature covers. Stalwart’s default signs five, and the two `List-Unsubscribe` headers are not among them. I sent them unsigned for the first six months of this newsletter, and unsigned they are useless. RFC 8058 requires them signed. That is one setting on the `DkimSignature` object, and it is the setting that will silently revert if you turn on automatic key rotation, so check it after any rotation.

The second was the hostname. Stalwart builds messages with its `mail-builder` crate, which reads `gethostname()` and falls back to the literal `localhost`, so every message I had ever sent

carried a `Message-ID` ending in `@localhost`. That is a well-known junk-folder smell. I set the system hostname and it went away.

Stalwart made the rest easy. It generates the DKIM keys, runs the DMARC and TLS reporting, serves the MTA-STS policy, and its admin page lists the exact records it wants published. The job is pasting them into your DNS provider and then checking, with a tool like internet.nl or a mail-tester, that they are what you think they are. Do that check before the first message, not after the first junk folder.

The Hacker News objection

The advice you will get, stated at full strength: never self-host email. Your IP shares a reputation with whoever had it before you. Microsoft will silently drop your mail and never tell you why. The blocklists are run by volunteers with no appeals process. One misconfiguration and you are on a list for months. Gmail changes its bulk-sender rules every year. You will spend your weekends on it, and the day you stop paying attention is the day your mail stops arriving.

Every one of those was true, and some are still true some of the time.

It's not that hard anymore to self-host email. There are so many people on Hacker News, for example, who say you should never attempt something like that, but so far it's been very good.

What changed is that the work moved into the server. Ten years ago "everything in order" meant Postfix, Dovecot, OpenDKIM, OpenDMARC, a Let's Encrypt hook and a policy daemon, each configured separately and each able to drift. Stalwart is one Rust binary that does SMTP, IMAP, JMAP, DKIM signing, ARC, DMARC and TLS reporting, MTA-STS and the admin UI, with its configuration in RocksDB. And the standards themselves have settled: SPF, DKIM, DMARC, MTA-STS, DANE and DNSSEC have not changed shape in years, and a receiver that sees all of them and a matching PTR has very little reason left to distrust you. The table above is the whole job.

Three concessions, because the objection is not all wrong:

- **Volume gates are real.** Gmail shows its native unsubscribe button only to bulk senders, roughly 5,000 messages a day to its addresses, so one-click is implemented and signed here and I will never see it work on my own mail.
- **The list is short.** A handful of readers is not a stress test of reputation. I will know more after a year of issues.
- **Rate limiting is still not finished.** The Workers bindings stop bursts and nothing slower, and the WAF rule that would stop a patient attacker is still on the list. (Since the move: nginx in front and a limiter in the process, keyed on the address and on the typed email.)

I know how to set it up for a company now, in case anyone needs self-hosted email and wants to get off Microsoft or Google.

That last sentence is the one I did not expect to write in February. Sovereignty over your own data has become a live question in the EU this year, and I wrote about the same trend on the model side in the Codex fork post: the customer who can leave is the customer who is fine. Mail is the oldest federated system we have. It was designed so that anyone can run a node, and after a decade in which that stopped being practical, it is practical again.

What this costs

Component	Cost
Cloudflare Pages	Free
Cloudflare Worker	Free, 100k requests a day
Stalwart on a Hetzner VPS	About $6 a month, shared with mail, auth and a secrets vault
Domain	About $10 a year

Mailchimp is $13 a month for 500 subscribers, ConvertKit is $29, Substack takes 10% of anything paid. This blog publishes twice a month to people who asked for it, and none of those platforms would do anything for it that the table above does not. The comparison is not close.

What is missing

- **Analytics.** No open tracking, no click tracking. I do not want it, but you might.
- **Bounce handling.** An address that hard-bounces stays on the list. The send log has the raw material and the rule is not written.
- **Scheduling.** I run one command when I want to send. There is no cron.
- **A list longer than 45.** See above.
- **A pretty editor.** I write markdown and the template is Rust. That one is a feature.

Workers-rs tips

The Worker is gone from this site, but these cost me time and are still true on `worker` 0.8 if you run one:

- **Register GET before POST for the same path.** The unsubscribe page and the unsubscribe handler share `/api/unsubscribe`, and registration order decided which one matched.
- `Headers` **methods take** `&self`**.** You do not need `let mut headers`.

- `strip = true` **breaks** `worker-build`**.** It removes the wasm `target_features` section that wasm-bindgen reads to detect reference types, and the build dies with "externref table required". Use `strip = "debuginfo"`; the artifact is the same size because wasm-opt drops the names anyway.

What I'd tell you

Publish every record in the table before the first message. Then check them from outside. The junk folder is where you find out you skipped one.

Sign the `List-Unsubscribe` **headers.** They do nothing unsigned, and nothing will tell you.

Double opt-in from day one, and keep the pending state in the link. You do not need a table for it, and a list you cannot prove consent for is a list you cannot mail.

Send one message per reader. It is the only way to give each reader a link that knows who they are, and everything downstream of that, from one-click to a send log, gets simpler.

Measure your rate limiter. I sent 18 sequential requests through a 15-per-minute limit and got zero 429s. The description said otherwise.

The service that replaced the Worker is on GitHub under `newsletter/`, the move is written up, and the box at the bottom of this page is the live system. If your company wants off Microsoft or Google and needs someone who has done it, write to me.

References

- Bathelt, H., Malmberg, A., & Maskell, P. "Clusters and knowledge: local buzz, global pipelines and the process of knowledge creation". *Progress in Human Geography*, vol. 28, no. 1, pp. 31-56, 2004. doi:10.1191/0309132504ph469oa
- Granovetter, M. S. "The Strength of Weak Ties". *American Journal of Sociology*, vol. 78, no. 6, pp. 1360-1380, 1973. doi:10.1086/225469